

From Chaos to Clarity: Cleaning and Analyzing Dirty Restaurant Sales Data Using R

Moshood A. Garuba

2025-05-05

Background:

In the bustling world of restaurants, daily sales data is a goldmine—but only if it's clean and structured. Unfortunately, real-life data is rarely pristine. Missing values, inconsistencies, and incomplete entries are the norm rather than the exception. This project tackles one such challenge using a “dirty” restaurant sales dataset containing 17,534 transactions across starters, main dishes, desserts, drinks, and side dishes.

The Problem:

The presence of dirt in data—such as missing categories, inconsistent entries, and incomplete transaction details—often makes analysis unreliable or impossible. Businesses that fail to clean and understand their data risk losing valuable insights that could improve operations, menu offerings, or customer satisfaction.

Data Strategy:

- Data Importation
- Data Cleaning
- Exploratory Data Analysis
- Statistical Modeling and Inference

Expected Outcome:

By the end of this project, the cleaned dataset will offer a reliable foundation for future analysis and modeling. Business insights—such as what items are most often paired, what times of day generate the most revenue, or what types of sales are prone to missing info—will be clearly visualized and discussed.

Data Importation and Loading of Libraries

```
# Load required libraries  
library(tidyverse)  
library(lubridate)  
library(readr)
```

```
library(janitor)
library(dplyr)
library(kableExtra)
library(naniar)
library(lubridate)
library(dlookr)
library(patchwork)
library(gt)
library(gtsummary)
library(report)
```

```
mydata <- read.csv("restaurant_sales_data.csv")
```

```
#checking the names of the variables
names(mydata)
```

```
## [1] "Order.ID"      "Customer.ID"    "Category"       "Item"
## [5] "Price"         "Quantity"       "Order.Total"    "Order.Date"
## [9] "Payment.Method"
```

```
#Checking the variable structure
str(mydata)
```

```
## 'data.frame': 17534 obs. of 9 variables:
## $ Order.ID : chr "ORD_705844" "ORD_338528" "ORD_443849" "ORD_630508" ...
## $ Customer.ID : chr "CUST_092" "CUST_021" "CUST_029" "CUST_075" ...
## $ Category : chr "Side Dishes" "Side Dishes" "Main Dishes" "Drinks" ...
## $ Item : chr "Side Salad" "Mashed Potatoes" "Grilled Chicken" "" ...
## $ Price : num 3 4 15 NA 12 18 4 15 6 14 ...
## $ Quantity : num 1 3 4 2 4 5 5 3 2 5 ...
## $ Order.Total : num 3 12 60 5 48 90 20 45 12 70 ...
## $ Order.Date : chr "2023-12-21" "2023-05-19" "2023-09-27" "2022-08-09" ...
## $ Payment.Method: chr "Credit Card" "Digital Wallet" "Credit Card" "Credit Card" ...
```

```
# Dimension
dim(mydata)
```

```
## [1] 17534 9
```

```
#Checking the first few observations
head(mydata) %>% kable()
```

Order.ID	Customer.ID	Category	Item	Price	Quantity	Order.Total	Order.Date	Payment.Method
ORD_705844	CUST_092	Side Dishes	Side Salad	3	1	3	2023-12-21	Credit Card
ORD_338528	CUST_021	Side Dishes	Mashed Potatoes	4	3	12	2023-05-19	Digital Wallet
ORD_443849	CUST_029	Main Dishes	Grilled Chicken	15	4	60	2023-09-27	Credit Card

Order.ID	Customer.ID	Category	Item	Price	Quantity	Order.Total	Order.Date	Payment.Method
ORD_630508	CUST_075	Drinks		NA	2	5	2022-08-09	Credit Card
ORD_648269	CUST_031	Main Dishes	Pasta Alfredo	12	4	48	2022-05-15	Cash
ORD_381680	CUST_031	Main Dishes	Salmon	18	5	90	2022-07-20	Digital Wallet

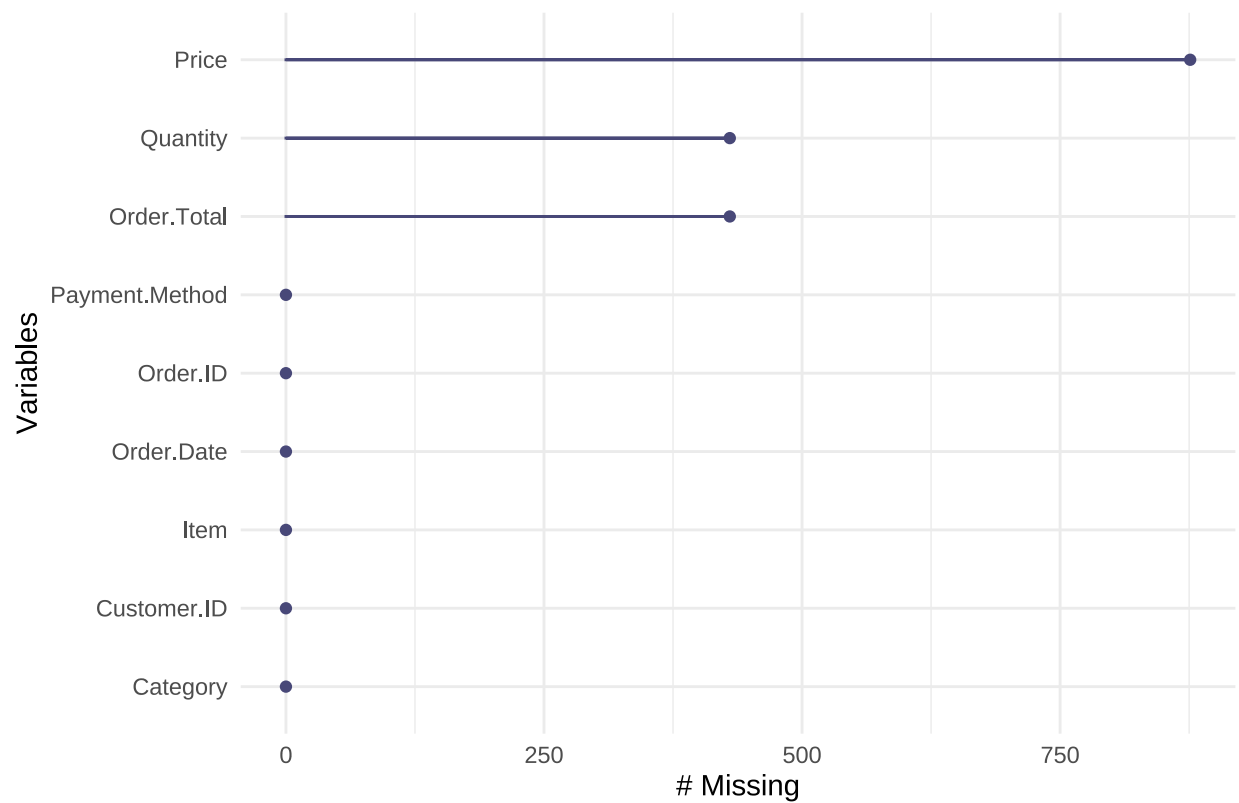
```
#Checking the last few observations
tail(mydata) %>% kable()
```

	Order.ID	Customer.ID	Category	Item	Price	Quantity	Order.Total	Order.Date	Payment.Method
17529	ORD_721774	CUST_006	Side Dishes	Mashed Potatoes	4	5	20	2022-04-22	Digital Wallet
17530	ORD_320102	CUST_021	Drinks		NA	4	4	2023-12-23	Cash
17531	ORD_974108	CUST_069	Desserts	Ice Cream	5	3	15	2023-12-18	Cash
17532	ORD_108324	CUST_050	Desserts	Ice Cream	5	4	20	2022-05-20	Digital Wallet
17533	ORD_612604	CUST_073	Side Dishes	Mashed Potatoes	4	2	8	2022-01-27	Digital Wallet
17534	ORD_680707	CUST_006	Desserts		7	4	28	2023-01-07	

Data Cleaning:

```
mydata %>%
  gg_miss_var()+
  labs(title = "Visaulization of the Missing variable")
```

Visualization of the Missing variable



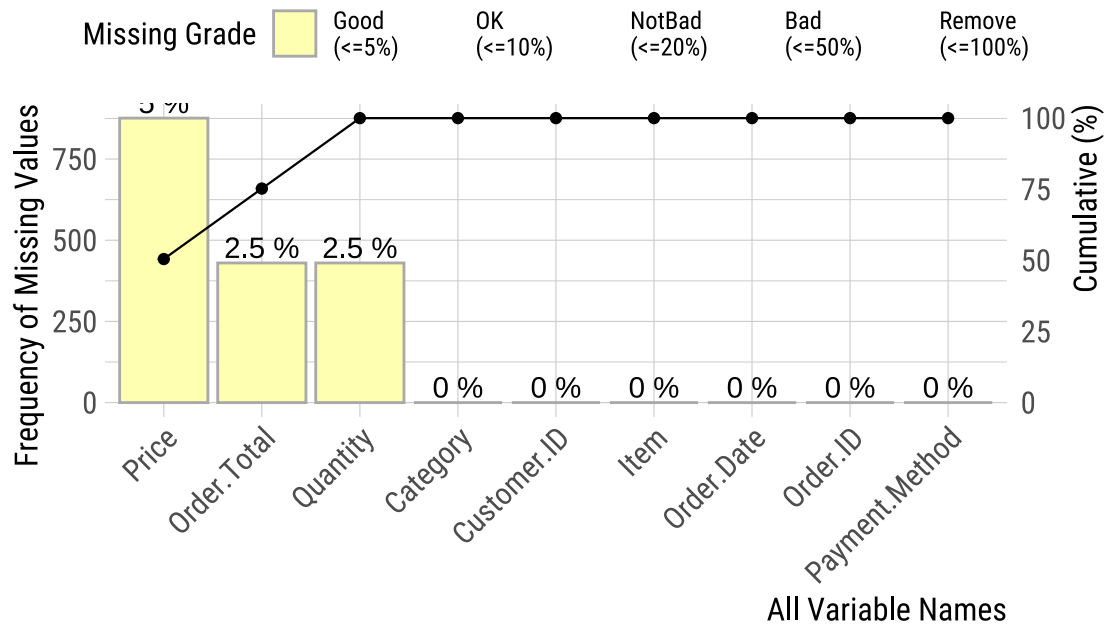
```
sum(!complete.cases(mydata)) == T)
```

```
## [1] 876
```

876 observations are missing in the dataset. We need to check the pattern of missing and develop a strategy for adjusting the missingness either by omission or imputation method using some machine learning algorithms.

```
plot_na_pareto(mydata)
```

Pareto chart with missing values



Observations: From the Pareto na missing chart is show that the missing is fair and we can impute and fill the missing observations using machine learning algorithms called **Predictive mean matching**.

```
# Step 1: Perform multiple imputation
imputed_data <- mice::mice(mydata, method = "pmm")
```

```
##
## iter imp variable
## 1 1 Price Quantity Order.Total
## 1 2 Price Quantity Order.Total
## 1 3 Price Quantity Order.Total
## 1 4 Price Quantity Order.Total
## 1 5 Price Quantity Order.Total
## 2 1 Price Quantity Order.Total
## 2 2 Price Quantity Order.Total
## 2 3 Price Quantity Order.Total
## 2 4 Price Quantity Order.Total
## 2 5 Price Quantity Order.Total
## 3 1 Price Quantity Order.Total
## 3 2 Price Quantity Order.Total
## 3 3 Price Quantity Order.Total
## 3 4 Price Quantity Order.Total
## 3 5 Price Quantity Order.Total
## 4 1 Price Quantity Order.Total
## 4 2 Price Quantity Order.Total
## 4 3 Price Quantity Order.Total
```

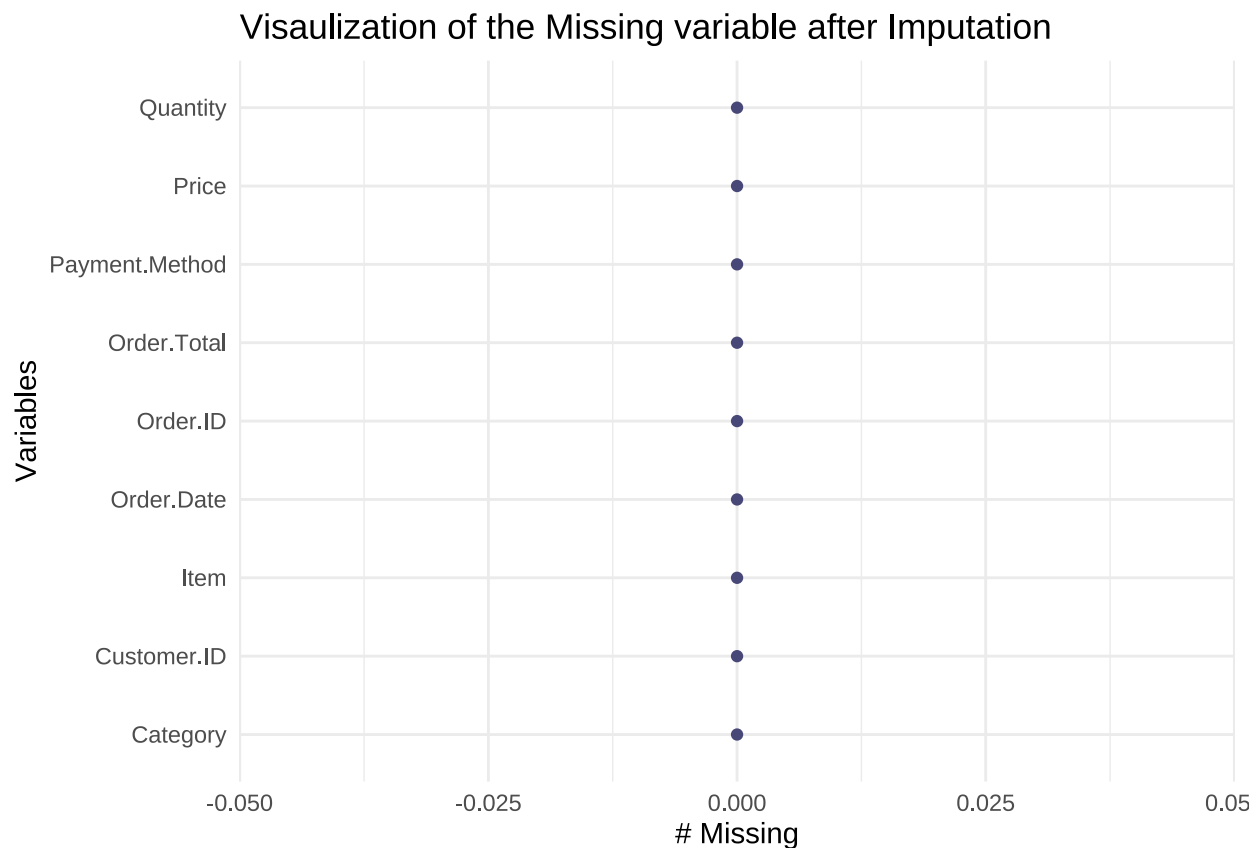
```
## 4 4 Price Quantity Order.Total
## 4 5 Price Quantity Order.Total
## 5 1 Price Quantity Order.Total
## 5 2 Price Quantity Order.Total
## 5 3 Price Quantity Order.Total
## 5 4 Price Quantity Order.Total
## 5 5 Price Quantity Order.Total
```

```
# Step 2: Extract the completed dataset (e.g., first imputation)
clean_data <- mice::complete(imputed_data, 1)

dim(clean_data)
```

```
## [1] 17534      9
```

```
gg_miss_var(clean_data)+
  labs(title = "Visaulization of the Missing variable after Imputation")
```

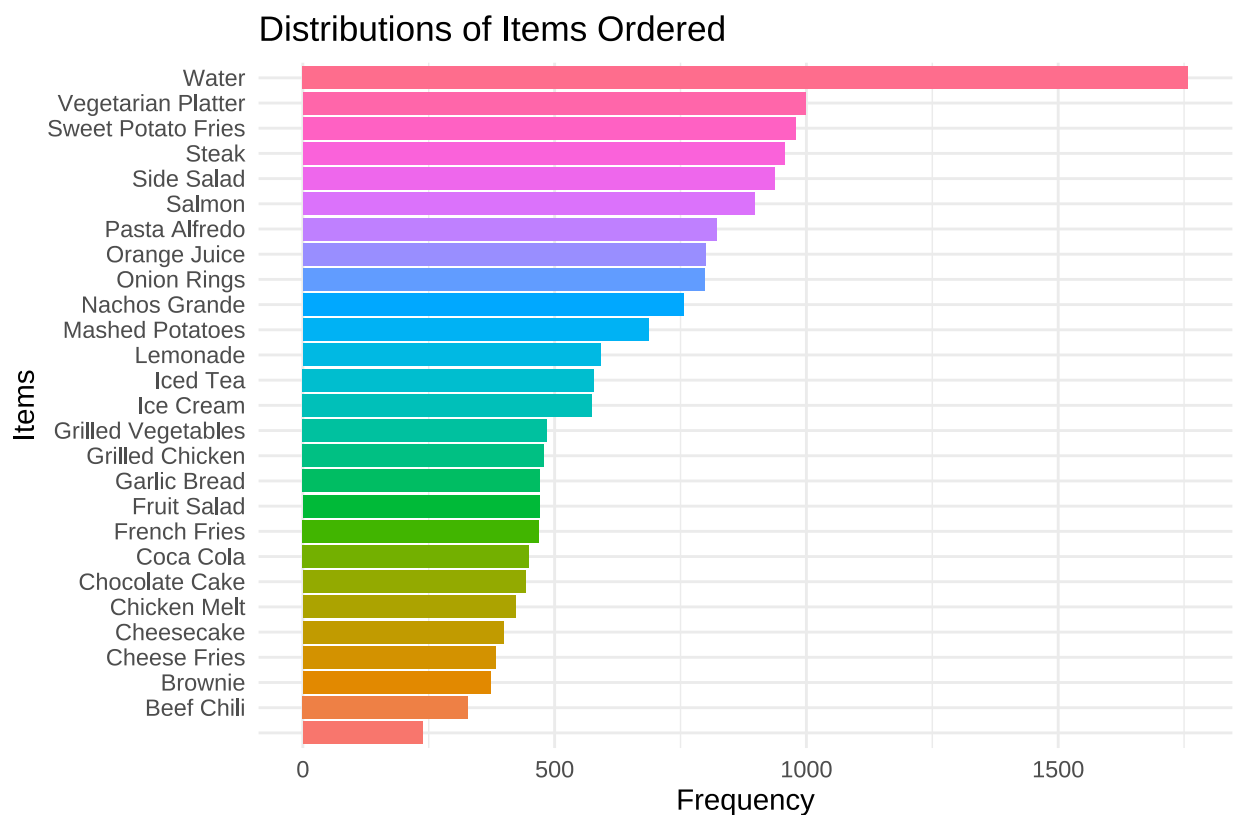


Observation: The only data problem is missing observation which has been corrected using **Predictive mean matching techniques** embended in `mice()` function and `mice` package.

Data Analysis:

Univariate analysis:

```
clean_data %>%  
  group_by(Item) %>%  
  summarise(n = n()) %>%  
  ggplot(aes(y = Item, x = sort(n))) +  
  geom_bar(stat = "identity", aes(fill = Item), show.legend = F) +  
  theme_minimal() +  
  labs(title = "Distributions of Items Ordered",  
        y = "Items", x = "Frequency", caption = "Source: Kaggle")
```



Source: Kaggle

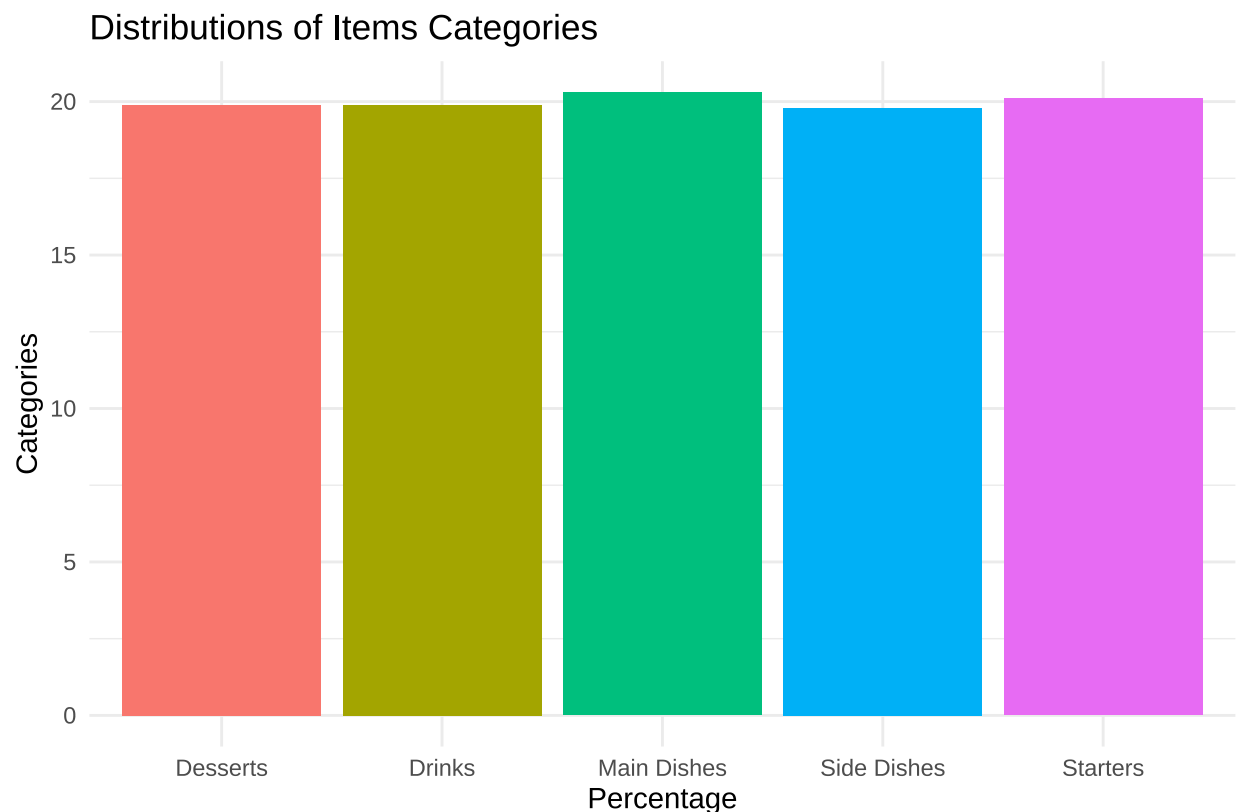
```
clean_data %>%  
  group_by(Category) %>%  
  summarise(n = n()) %>%  
  mutate(Percentage = round(n/sum(n),3)*100) %>% kable()
```

Category	n	Percentage
Desserts	3496	19.9
Drinks	3488	19.9
Main Dishes	3551	20.3

Category	n	Percentage
Side Dishes	3472	19.8
Starters	3527	20.1

Obseveration: The distribution of items across categories in the restaurant sales data is relatively balanced. Main Dishes have the highest count, making up 20.3% of all items sold ($n = 3,551$), closely followed by Starters (20.1%) and Desserts (19.9%). Drinks and Side Dishes account for 19.9% and 19.8% respectively. The differences among the categories are minimal, suggesting that customer demand is fairly evenly spread across all five item types, with a slight preference for main dishes.

```
clean_data %>%
  group_by(Category) %>%
  summarise(n = n()) %>%
  mutate(Percentage = round(n/sum(n),3)*100) %>%
  ggplot(aes(x = Category, y = Percentage ))+
  geom_bar(stat = "identity", aes(fill = Category), show.legend = F)+
  theme_minimal()+
  labs(title = "Distributions of Items Categories",
       y = "Categories", x = "Percentage", caption = "Source: Kaggle")
```



Source: Kaggle

```
clean_data %>%
  summarytools::descr(Price, stat = "common")
```

Descriptive Statistics


```
## clean_data$Price
## N: 17534
##
##              Price
## -----
##      Mean      6.60
##    Std.Dev    4.86
##      Min      1.00
##     Median     5.00
##      Max     20.00
##    N.Valid  17534.00
##    Pct.Valid  100.00
```

Observation: The average price of items sold is **\$6.60**, with a **standard deviation of \$4.85**, indicating moderate variability in pricing. The **minimum price** observed is **\$1.00**, while the **maximum** reaches **\$20.00**, showing a wide range of item prices. The **median price** is **\$5.00**, which is lower than the mean, suggesting a slight right-skew in the price distribution (i.e., a few higher-priced items pulling the average upward). All **17,534 observations** are valid, making up **100%** of the data, with no missing values for price.

```
clean_data %>%
  summarytools::descr(Quantity, stat = "common")
```

```
## Descriptive Statistics
## clean_data$Quantity
## N: 17534
##
##              Quantity
## -----
##      Mean      3.01
##    Std.Dev    1.42
##      Min      1.00
##     Median     3.00
##      Max      5.00
##    N.Valid  17534.00
##    Pct.Valid  100.00
```

Observation: The average quantity of items ordered is **3.01**, with a **standard deviation of 1.42**, indicating a relatively consistent ordering pattern. The **minimum quantity** is **1**, and the **maximum** is **5**, suggesting that customers typically order between one and five items at a time. The **median quantity** is **3**, which aligns closely with the mean, indicating a fairly symmetrical distribution. All **17,534 observations** are valid, representing **100%** completeness for the **Quantity** variable.

```
clean_data %>%
  summarytools::descr(Order.Total, stat = "common")
```

```
## Descriptive Statistics
## clean_data$Order.Total
## N: 17534
##
##              Order.Total
## -----
##      Mean      19.88
```

```
##          Std.Dev      18.78
##          Min         1.00
##          Median      15.00
##          Max        100.00
##          N.Valid     17534.00
##          Pct.Valid    100.00
```

Observation: The average order total is **\$19.90**, with a **standard deviation of \$18.83**, indicating considerable variability in how much customers spend per transaction. The **minimum order total** is **\$1.00**, while the **maximum** reaches **\$100.00**, showing a wide range of spending behavior. The **median** order total is **\$15.00**, which is lower than the mean, suggesting a slight right-skew in the data due to some high-value orders. All **17,534 observations** are valid, reflecting **100%** data completeness for the `Order.Total` variable.

Bivariate Analysis:

```
clean_data %>%
  group_by(Item) %>%
  summarise(
    n = n(),
    `Total Quantity` = sum(Quantity, na.rm = TRUE)
  ) %>%
  mutate(`Average Quantity` = round(`Total Quantity` / n, 2)) %>%
  top_n(10, wt = `Total Quantity`) %>% kable()
```

Item	n	Total Quantity	Average Quantity
	1758	5210	2.96
Chocolate Cake	798	2348	2.94
Coca Cola	756	2282	3.02
French Fries	897	2721	3.03
Grilled Chicken	822	2492	3.03
Ice Cream	936	2929	3.13
Mashed Potatoes	799	2374	2.97
Pasta Alfredo	998	3094	3.10
Side Salad	978	2932	3.00
Water	956	2899	3.03

```
clean_data %>%
  group_by(Item) %>%
  summarise(
    n = n(),
    `Ordered Quantity` = sum(Order.Total, na.rm = TRUE)
  ) %>%
  mutate(`Average Quantity` = round(`Ordered Quantity` / n, 2)) %>%
  top_n(10, wt = `Ordered Quantity`) %>% kable()
```

Item	n	Ordered Quantity	Average Quantity
	1758	33819	19.24
Chicken Melt	442	10640	24.07
Chocolate Cake	798	14088	17.65
French Fries	897	10884	12.13
Grilled Chicken	822	37380	45.47
Ice Cream	936	14645	15.65
Pasta Alfredo	998	37128	37.20
Salmon	422	22608	53.57
Steak	574	35320	61.53
Vegetarian Platter	382	15708	41.12

```
clean_data %>%
  group_by(Category) %>%
  summarise(
    n = n(),
    `Total Quantity` = sum(Quantity, na.rm = TRUE)
  ) %>%
  mutate(`Average Quantity` = round(`Total Quantity` / n, 2)) %>%
  top_n(10, wt = `Total Quantity`) %>% kable()
```

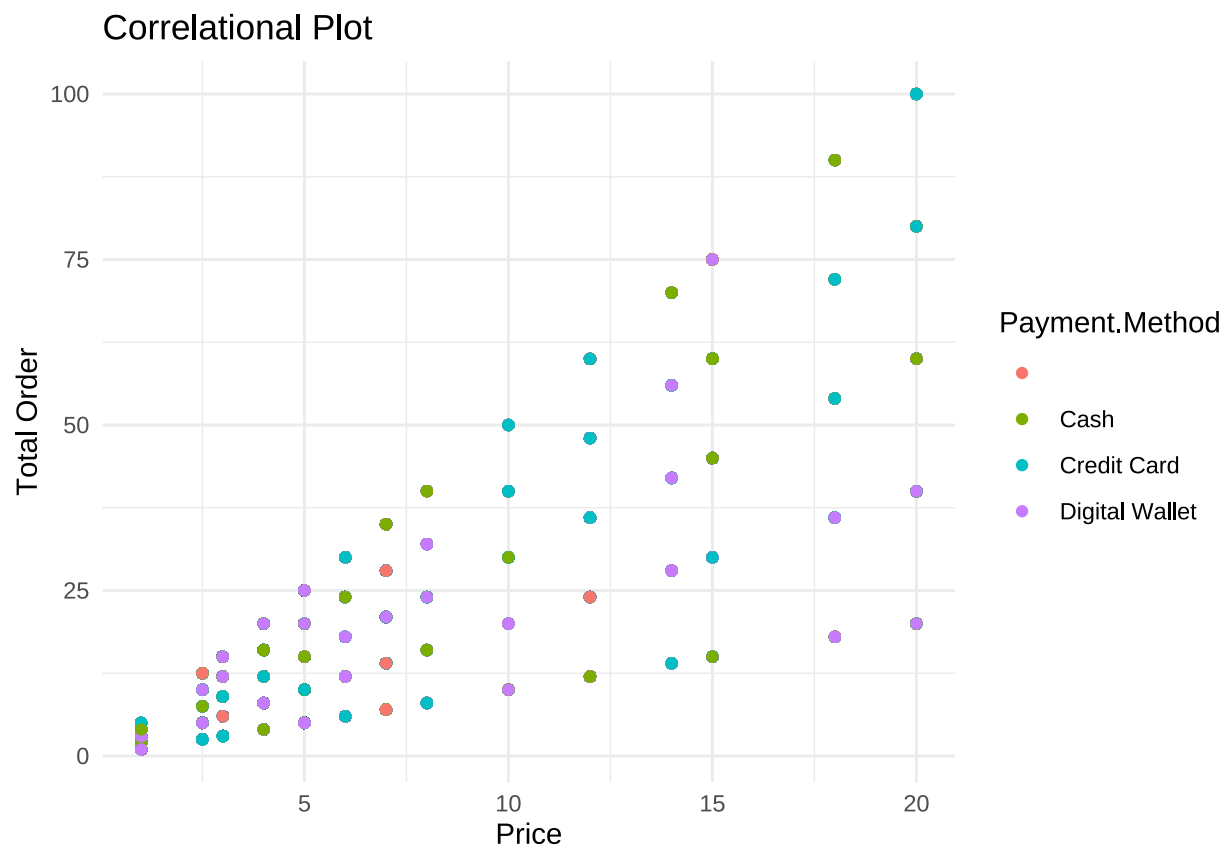
Category	n	Total Quantity	Average Quantity
Desserts	3496	10511	3.01
Drinks	3488	10510	3.01
Main Dishes	3551	10810	3.04
Side Dishes	3472	10372	2.99
Starters	3527	10581	3.00

```
clean_data %>%
  group_by(Category) %>%
  summarise(
    n = n(),
    `Ordered Quantity` = sum(Order.Total, na.rm = TRUE)
  ) %>%
  mutate(`Average Quantity` = round(`Ordered Quantity` / n, 2)) %>%
  top_n(10, wt = `Ordered Quantity`) %>% kable()
```

Category	n	Ordered Quantity	Average Quantity
Desserts	3496	58646.5	16.78
Drinks	3488	24232.5	6.95
Main Dishes	3551	162373.0	45.73
Side Dishes	3472	41908.0	12.07
Starters	3527	61340.5	17.39

Statistical Modeling and Inference

```
clean_data %>%
  ggplot(aes(x = Price, y = Order.Total))+
  geom_point(aes(colour = Payment.Method))+
  labs(title = "Correlational Plot", y = "Total Order")+
  theme_minimal()
```



```
model<-lm(Order.Total~Price, data = clean_data)
```

```
summary(model)
```

```
##
## Call:
## lm(formula = Order.Total ~ Price, data = clean_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -40.475  -5.963  -0.024   5.946  39.525
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.12670    0.14830  -0.854   0.393
## Price        3.03006    0.01809 167.517 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##  
## Residual standard error: 11.65 on 17532 degrees of freedom  
## Multiple R-squared:  0.6155, Adjusted R-squared:  0.6155  
## F-statistic: 2.806e+04 on 1 and 17532 DF,  p-value: < 2.2e-16
```

A linear regression model was fitted using ordinary least squares (OLS) to predict `Order.Total` based on `Price`. The model revealed a statistically significant and substantial relationship, explaining 62% of the variance in order totals ($\mathbf{R}^2 = \mathbf{0.62}$, $F(1, 17,532) = 28,132.81$, $p < .001$). The intercept of the model was -0.23 (95% CI [-0.52, 0.07]), which was not statistically significant ($t = -1.52$, $p = 0.128$), indicating no meaningful prediction when price is zero. However, the effect of `Price` was highly significant and positive ($\beta = \mathbf{3.05}$, 95% CI [3.01, 3.08], $t = 167.73$, $p < .001$), with a standardized beta of $\mathbf{0.78}$ (95% CI [0.78, 0.79]), suggesting that higher prices are strongly associated with higher order totals. Overall, the model demonstrates that price is a strong and reliable predictor of total sales.